**The recommended architecture for generating trading signals involves combining two distinct data pipelines for robust and reliable performance: an Official Streaming API** and a **Live Search Polling Scraper**. This dual-source approach, common in social-signal trading systems, ensures continuous data flow even when API limits are encountered.**1. Official Streaming API (Reliable Real-Time Data)**

This pipeline uses the **Filtered Stream endpoint from X** (formerly Twitter) to deliver relevant tweets in real time.**Implementation Steps:**

1. **Create Streaming Rules:** Define the criteria for tweets you want to receive (e.g., token tickers, project names, contract addresses, hashtags).
    - *Example Rules:* `(sol OR solana) lang:en`, `$BONK OR bonk`, a contract address `0x...`
    - *Adding Rules via API:* `POST [https://api.x.com/2/tweets/search/stream/rules](https://api.x.com/2/tweets/search/stream/rules)`
    - *Example Payload:*
    - {
    - "add": [
    - {"value": "solana OR $SOL lang:en"},
    - {"value": "bonk OR $BONK"}
    - ]
    - }
2. **Connect to the Streaming Endpoint:** Connect to `[https://api.x.com/2/tweets/search/stream](https://api.x.com/2/tweets/search/stream)` to receive matching tweets within seconds of posting.
    - *Python Snippet:* A script using the `requests` library with a Bearer Token can maintain a persistent connection, processing each tweet line as it arrives.
3. **Send Data to Your Pipeline:** The real-time tweets should be processed through a scalable architecture:
    - **Architecture Flow:** Stream API → Tweet Parser → Message Queue (Redis/Kafka) → ML Model → Trading Signals.

**2. Live Search Timeline Scraper (Fast Fallback)**

This acts as a backup data source for situations where API rate limits are hit. Instead of streaming, it repeatedly queries the latest search results on the platform.**Implementation Details:**

- **Mechanism:** Periodically (e.g., every 3–5 seconds) query a live search URL (e.g., `[https://twitter.com/search?q=solana&f=live](https://twitter.com/search?q=solana&f=live)`).

- **Basic Python Implementation (HTML Scraper):** Using libraries like `requests` and `BeautifulSoup4`, the script fetches the search page, parses the HTML, finds tweet elements (`<article>`), and prints the content.
- **Advanced Optimization:** For greater stability, use specialized libraries like **snscrape** or **Playwright** instead of fragile manual HTML parsing.

### 3. Deduplicate and Process Tweets

Since data is collected from two sources, it's essential to remove duplicates based on a unique identifier like the `tweet_id`.

- **Deduplication Logic:** Check if `tweet_id` is present in a persistent storage (e.g., Redis, PostgreSQL, or MongoDB) before processing.

### 4. Recommended Production Pipeline (AI Trading Bot)

The robust architecture for a high-performance AI trading bot integrates both sources:

```
graph TD
  A[X Stream API] --> C(tweet collector);
  B[search scraper] --> C;
  C --> D(message queue);
  D --> E(feature builder);
  E --> F(ML model);
  F --> G(trade engine);
```

 **Pro Tip: Feature Engineering for Pump Detection**

For your pump-detection model, track these key features for every tweet:

- `tweet_time`
- `followers_count`
- `retweet_count`
- `like_count`
- `token_mentions`
- `contract_address`